School of Innovative Technologies and Engineering (SITE)

BSc (Hons) Software Engineering

Data Structures and Algorithms - PROG2101C

Name:RANAIVOMAMPIONONA FENOHASINA MIHANTA Johane
Student ID: 2303_26116 ***COHORT:*** BSE23AFT(1)

Calculator Project Report

1-Introduction:

Description of the Assignment and Objectives

The objective of this assignment was to develop a calculator application capable of performing various mathematical operations, including basic arithmetic, trigonometric functions, logarithms, and square roots. The project required the implementation of a graphical user interface (GUI) using Java Swing and the utilization of data structures and algorithms for efficient expression evaluation.

The primary goals included:
- Designing a user-friendly calculator interface.
- Implementing a reliable algorithm for parsing and evaluating mathematical expressions. - Utilizing appropriate data structures to handle the complexity of mathematical operations.

2-Design:

Layout(s) of the Calculator

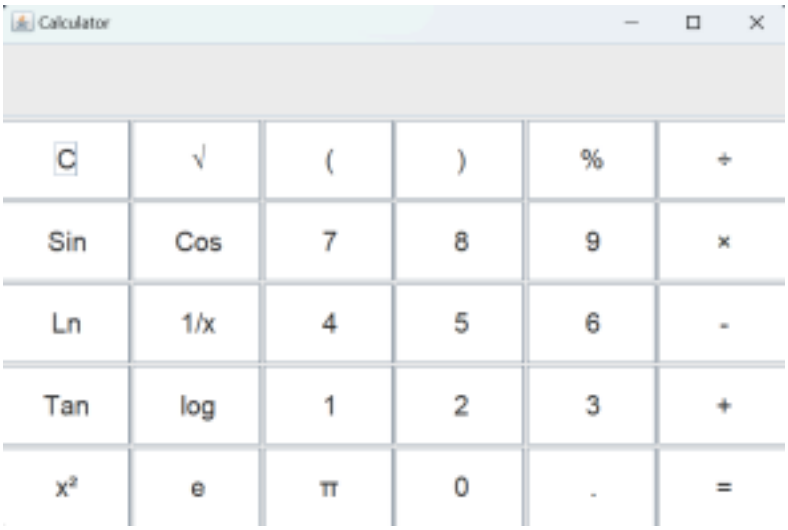
The calculator features a grid-based layout, with a display area and a grid of buttons representing various mathematical functions and operations. The layout is designed to be intuitive and user-friendly, featuring:

- **Display Area**: A `JTextField` at the top of the frame for showing the current expression and result.
- **Buttons:** The calculator features a set of buttons organized in a GridLayout, comprising numerical digits (0-9), arithmetic operators (+, -, ×, ÷), mathematical functions (sin, cos, log), and special functions (square root, reciprocal). The use of Unicode characters, such as √ and ×, eliminates the need to load images for these symbols.(source: stack overflow)

List:

- (x^2) power of 2 : x\u00B2

- "Square Root: \u221A"



Data Structure(s) Used

a. Stack:

- Purpose: Used for managing operators and operands during the infix-to-postfix conversion and for evaluating the postfix expression.
- Implementation: Java's built-in `Stack` class.
- Operations: Push, pop, and peek.

b. StringBuilder:

- Purpose: Efficiently constructs the mathematical expression and result display.
- Implementation: Java's `StringBuilder` class.

Algorithms

a. Infix to Postfix Conversion:

- Algorithm: Shunting-yard algorithm.
  **Shunting-yard:The Shunting-yard algorithm is a method for converting infix expressions to postfix expressions. It works by using a stack to store operators and operands, and then outputting the postfix expression.

- Description: Converts infix expressions (like "3 + 4") to postfix expressions (like "3 4 +") to facilitate easier evaluation.
- Steps:
  . Read the infix expression from left to right.
  . Use a stack to manage operators and parentheses.
  . Output the operands and operators in postfix order.

b. Postfix Evaluation:

- Algorithm: Evaluates postfix expressions using a stack.

- Description: Computes the result of the postfix expression by processing each token and applying operators to operands.
- Steps:
  . Read the postfix expression token by token.
  . Use a stack to handle operands and intermediate results.
  . Apply operators as they are encountered and push results back onto the stack.

3-Testing:

Test Data

- Basic Arithmetic Operations:
  Expression: ((5+6)*30)/300
  Expected Result: 1.10

- Trigonometric Functions:
  Expression: tan((72+6)/2)
  Expected Result ~0.81

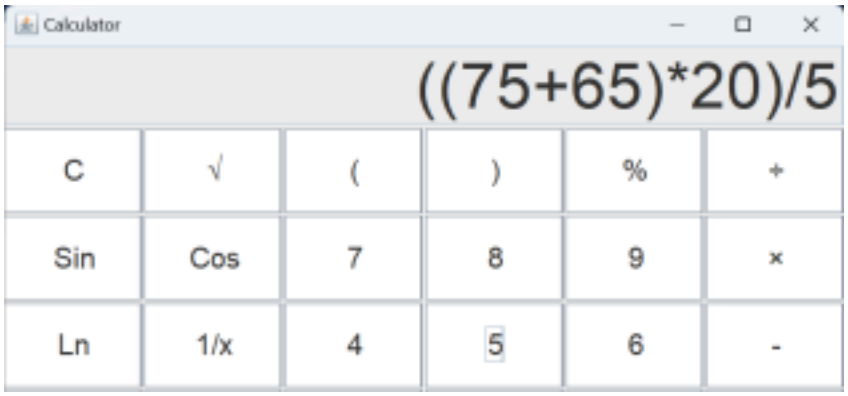
- Logarithmic Functions:
  Expression: log(100)
  Expected Result: 2.00

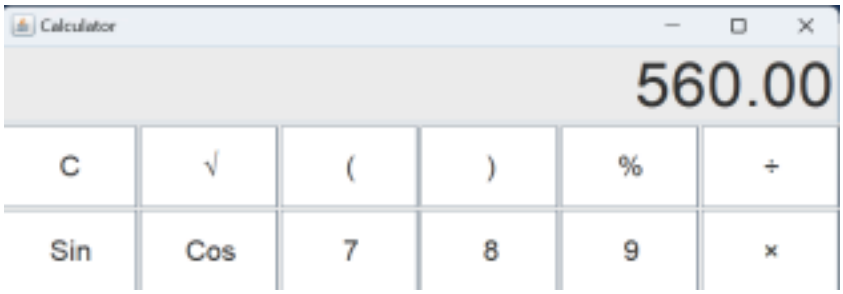
- Special Functions:
  Expression: `sqrt(16)`
  Expected Result: 4.00

- Error Handling:
  Expression: 1 / 0
  Expected Result: `Error` (Division by zero)

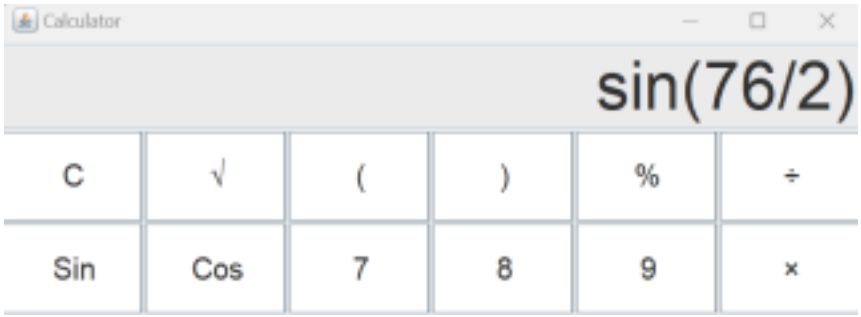
Screenshots

- Basic Operation:

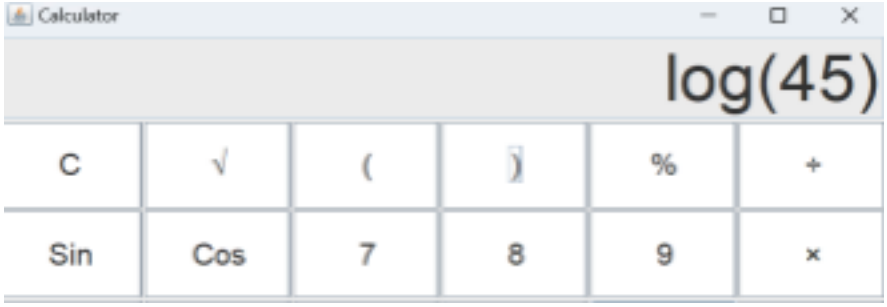




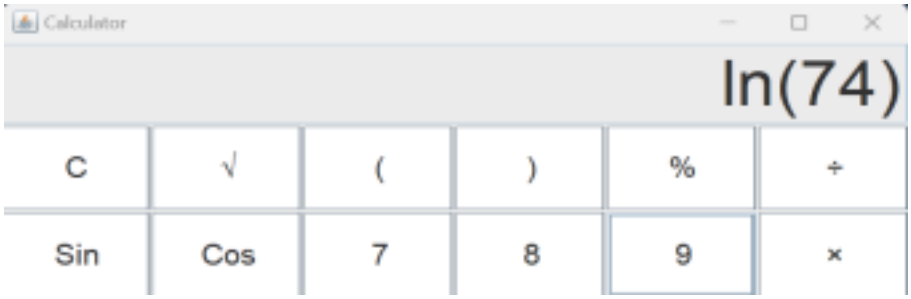
- Trigonometric Function:



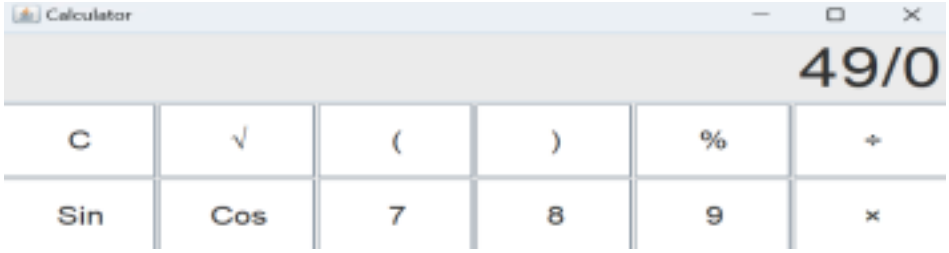
- Logarithmic Function:

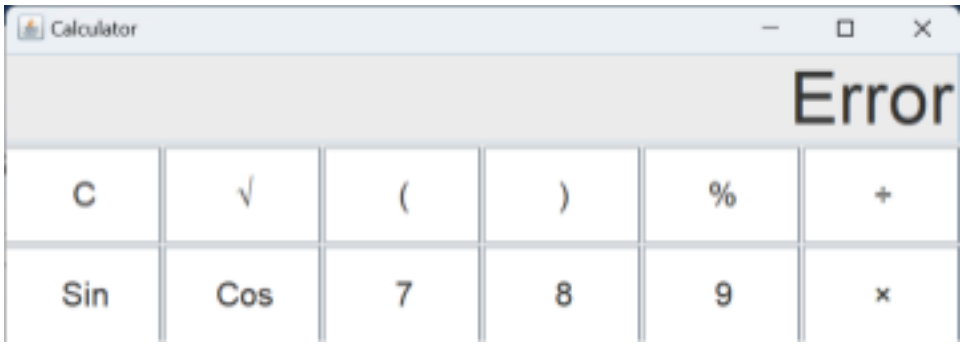


- Exponential Function:



- Error Handling:





<u>How the program works:</u>
Let's go through an example to illustrate how the calculator works.

Suppose we want to calculate the expression: `2 + 3 * 4`

Step 1: Button Clicks

We click the following buttons in sequence:

- 2
- +
- 3
- *
- 4
- =

Step 2: Expression Building

As we click each Button, the corresponding text is appended to the current expression

- 2 -> 2
- + -> 2 +
- 3 -> 2 + 3
- * -> 2 + 3 *
- 4 -> 2 + 3 * 4

The final expression is: `2 + 3 * 4`
Step 3: Infix to Postfix Conversion

The infixToPostfix method converts the infix expression to postfix notation:

`2 + 3 * 4` **->** `2 3 4 * +`

Step 4: Postfix Evaluation

The evaluatePostfix method evaluates the postfix expression:

- `2 3 4 * +` **->**
  1. Push `2` onto the stack: `[2]`
  2. Push `3` onto the stack: `[2, 3]`
  3. Push `4` onto the stack: `[2, 3, 4]`
  4. Pop `4` and `3` from the stack, multiply them, and push the result: `[2, 12]`
  5. Pop `2` and `12` from the stack, add them, and push the result: `[14]` -
  The final result is(display on the screen): `14`

4-Conclusion

The project successfully implements a calculator that can evaluate complex mathematical expressions using a GUI and appropriate data structures. The use of stacks for infix-to-postfix conversion and postfix evaluation ensures that the calculator handles mathematical operations efficiently.
NB: the use of double is to have an approximation of the calculation

Key Points:
Efficiency:
The algorithms used (infix-to-postfix conversion and postfix evaluation) are efficient, with time complexity \(O(n)\), making the calculator responsive and accurate.
- Error Handling: Basic error handling mechanisms are in place to manage invalid operations such as division by zero.

Overall, the project meets the assignment objectives and demonstrates effective use of data structures and algorithms to create a functional and reliable calculator application.